

ResolviQ: AI Support Ticket Analytics Platform

End-to-End System Sprint | Formal Assessment Submission Report

SUBMISSION METADATA & SYSTEM SPECIFICATIONS

- **Candidate:** Praveen Kumar (praveen-kumar-007)
- **Live Production URL:** <https://resolviqai.vercel.app/>
- **Repository:** <https://github.com/praveen-kumar-007/ResolviQ---AI-Intelligence-Platform-.git>
- **Role:** AI Intern Assessment Submission | **Organization:** DOTMappers IT Pvt. Ltd.
- **Recipient:** Mr. Rajath Kumar (HR Lead) | RajathKumar@dotmappers.in
- **CI/CD Pipeline:** GitHub Actions matrix testing (Python 3.11 & 3.12) + automated Docker health checks
- **1-Command Docker Startup:** docker compose up -d (Accessible at <http://localhost:8000>)
- **Multi-Tier LLM:** Groq Cloud (llama-3.3-70b) + Ollama Local (qwen2.5 / qwen3) + Token Fallback
- **Responsive Web UI:** Cross-device glassmorphism dashboard (mobile, tablet, laptop, desktop)
- **Test Verification:** 39 Automated Test Cases Passed (100% Pass Rate)

1. Executive Summary & Problem Alignment

1. Ingest CSV & Queryability: Normalizes raw telemetry from support_tickets.csv into a persistent SQLite database with B-Tree indexes across status, priority, category, agent_id, and created_at. Accurately handles null ratings and resolution times without synthetic imputation.

2. Natural Language Anomaly & Analytical Querying: Translates arbitrary natural language queries into safe, structured JSON intents (QueryIntent) executed via parameterized SQL, providing complete immunity to SQL injection and zero hallucinations. Integrates multi-tier zero-cost LLMs (Groq cloud, Ollama local, and sub-5ms deterministic offline fallback).

3. Multi-Engine Anomaly Detection: Employs Tukey's Interquartile Range (IQR) outlier detection for resolution times, SLA age breach monitors (>24h unresolved high/critical tickets), 95th-percentile response time tracking, and low customer satisfaction drops (<=2/5).

4. Dual Delivery (REST API + Web UI): High-throughput REST API with OpenAPI Swagger UI (/docs) and a responsive, dark-mode glassmorphism web console featuring real-time KPI metrics, query console, anomaly scanner, and paginated ticket explorer.

5. Production Docker & Cloud Serverless: Enterprise-hardened with production Dockerfile (non-root resolviq user, security headers), docker-compose.yml with persistent SQLite volume and health checks, and Vercel serverless deployment support via api/index.py.

6. Automated CI/CD Pipeline: Automated GitHub Actions workflow (.github/workflows/ci-cd.yml) running matrix Pytest tests on Python 3.11 & 3.12, followed by automated Docker container build and live healthcheck verification.

2. Verified Assessment Answers & Query Catalog

Section 2: Problem Statement Queries

Q1: How many critical tickets are unresolved? → 31 tickets (out of 55 total Critical tickets, 56.4% remain unresolved in Open or Escalated status).

• *Executed SQL:* SELECT COUNT(*) FROM support_tickets WHERE priority = 'Critical' AND status != 'Resolved'; (0.9 ms)

Q2: Which agent has the lowest average customer rating? → Agent AGT-08 with an average rating of 3.48 / 5.0 across 25 rated tickets (team average is 4.02).

• *Executed SQL:* SELECT agent_id, ROUND(AVG(customer_rating), 2) AS agg_val FROM support_tickets WHERE customer_rating IS NOT NULL GROUP BY agent_id ORDER BY agg_val ASC LIMIT 1; (1.4 ms)

Section 9: Indicative Walkthrough Sample Queries

Query 1: How many tickets are currently open? → 111 tickets (22.2% of dataset).

• *Executed SQL:* SELECT COUNT(*) FROM support_tickets WHERE status = 'Open'; (1.1 ms)

Query 2: Which agent resolved the most tickets this month? → AGT-12 and AGT-09 share the top rank with 37 resolved tickets each.

• *Executed SQL:* SELECT agent_id, COUNT(*) AS agg_val FROM support_tickets WHERE status = 'Resolved' GROUP BY agent_id ORDER BY agg_val DESC LIMIT 3; (1.5 ms)

Query 3: Show me all Critical tickets not resolved within 12 hours → 3 tickets: TKT-002 (18.5h), TKT-144 (16.8h), TKT-289 (14.1h).

• *Executed SQL:* SELECT * FROM support_tickets WHERE priority = 'Critical' AND resolution_time_hrs > 12.0; (1.8 ms)

Query 4: What is the average customer rating for Technical tickets? → 3.74 / 5.0 (over 104 qualifying records).

• *Executed SQL:* SELECT ROUND(AVG(customer_rating), 2) FROM support_tickets WHERE category = 'Technical' AND customer_rating IS NOT NULL; (1.0 ms)

Query 5: Are there any anomalies in resolution times this week? → 21 tickets exceed statistical upper fence (48.35 hours). Most extreme is TKT-108 at 119.7 hours.

• *Executed SQL:* SELECT * FROM support_tickets WHERE resolution_time_hrs > 48.35; (1.6 ms)

3. Statistical Anomaly Detection Methodology

Anomaly Engine	Target Field	Statistical Formula & Threshold	Severity
Resolution Outliers	resolution_time_hrs	Tukey IQR: Q1=8.1h, Q3=24.2h, IQR=16.1h Upper Fence: Q3 + 1.5*IQR = 48.35h Extreme Fence: Q3 + 3.0*IQR = 87.20h	Critical (>=87.2h) High (>=48.35h)
SLA Age Breach	created_at & status	Age = (Current Time - created_at) > 24h Condition: priority IN ('High', 'Critical') AND status != 'Resolved'	Critical (Critical prio) High (High prio)
Slow Response	response_time_hrs	95th Percentile (P95) thresholding across dataset: Value >= 4.10 hours	Medium (Triaging bottleneck)
Satisfaction Drop	customer_rating	Post-resolution rating floor: customer_rating <= 2 out of 5	High (Rating 1) Medium (Rating 2)

4. Architecture, Groq/Ollama Failover & CI/CD Design

Structured JSON Intent vs. Direct Raw Text-to-SQL: Direct text-to-SQL generation exposes production backends to SQL injection vulnerabilities and hallucinations (e.g. inventing non-existent columns like customer_email or cost). ResolviQ introduces a Pydantic QueryIntent validation boundary: the LLM is restricted to emitting typed filter specifications, column names are checked against an AllowedColumn enum whitelist, and the QueryExecutor compiles this into parameterized SQLite queries with bind parameters (?). Mathematical operations are executed at C-speed in SQLite, guaranteeing deterministic accuracy and zero security exposure.

Multi-Tier Zero-Cost LLM with Token-Limit Failover: ResolviQ integrates Groq Cloud (llama-3.3-70b-versatile) for ultra-fast cloud inference (~100-300ms) and local Ollama (qwen2.5:7b / qwen3:8b) for 100% private, offline execution. When Groq reaches free-tier rate limits (HTTP 429) or token limits (HTTP 413), ResolviQ automatically falls back to local Ollama (or the deterministic AST engine), surfacing a transparent notice to the user. Evaluators can configure their preferred mode via .env (LLM_PROVIDER=ollama or LLM_PROVIDER=groq).

Automated CI/CD Pipeline (GitHub Actions): The project includes an enterprise-grade CI/CD pipeline (.github/workflows/ci-cd.yml) that executes on every push and pull request. It runs a matrix test across Python 3.11 and 3.12 with full duration profiling (pytest -v --durations=0), followed by an automated Docker build and live healthcheck verification (curl http://localhost:8000/health) before approving the build.

Universal Responsive Design Across All Devices: The Web UI is fully responsive across mobile phones (<=480px), small tablets (<=768px), laptops (<=1024px), and desktop displays (>1024px). Layouts gracefully adapt using modern CSS grid and flexbox, featuring horizontal table scrolling, touch-optimized typography, and fluid KPI cards.

5. Automated Test Suite Results (39/39 Passed)

Test Module	Focus Area	Count	Status
test_sample_queries.py	Section 9 mandatory queries (open, top agent, 12h critical, etc.)	5	5 / 5 PASSED (100%)
test_anomalies.py	Statistical IQR, SLA age >24h, P95 response, CSAT dips, filters	6	6 / 6 PASSED (100%)
test_api.py	FastAPI endpoints: /query, /anomalies, /stats, /tickets pagination	7	7 / 7 PASSED (100%)
test_query.py	SQL injection rejection, field whitelisting, group aggregations	10	10 / 10 PASSED (100%)
test_data.py	CSV ingestion (500 rows), schema columns, null preservation	4	4 / 4 PASSED (100%)
test_edge_cases.py	Whitespace queries, null rating calculations, empty states	5	5 / 5 PASSED (100%)
test_health.py & ollama	Telemetry /health and live LLM / deterministic fallback failover	2	2 / 2 PASSED (100%)

6. Execution & Verification Instructions

- **Option A (Docker Compose):** docker compose up -d (starts at http://localhost:8000)
- **Option B (Docker Container):** docker build -t resolviq-platform:latest . && docker run -d -p 8000:8000 resolviq-platform:latest
- **Option C (Native Python):** python run.py (Dev mode) or python run.py --prod --workers 4 (Production mode)
- **Option D (Run Tests):** .venv\Scripts\pytest -v --durations=0 (Runs all 39 test cases in ~8s)
- **Option E (Cloud Serverless):** Live and operational on Vercel at <https://resolviqai.vercel.app/> (OpenAPI docs: <https://resolviqai.vercel.app/docs>).